

Universidade Federal de Minas Gerais
Departamento de Ciência da Computação
Curso de Graduação em Ciência da Computação

Trabalho Prático 1 - Redes de Computadores

Batalha de Naves

Lucas Affonso Pires
Matrícula: 2023028420

Belo Horizonte, Outubro de 2025

1 Introdução

O trabalho implementa um pequeno protocolo de batalha entre um *server* e um *client* em C, trocando mensagens serializadas via TCP. A aplicação simula rodadas em que cada lado escolhe uma ação (Laser, Torpedo, Shields, Cloaking, Hyper Jump) e o servidor resolve o turno e envia o resultado. A comunicação usa mensagens de texto.

2 Desafios encontrados

Abaixo estão os principais problemas que surgiram durante o desenvolvimento:

- **Portabilidade entre POSIX e Windows:** o código originalmente usa headers POSIX (`arpa/inet.h`, `sys/socket.h`), que não existem no Windows nativo; Windows exige Winsock (`winsock2.h`, `ws2tcpip.h`) e inicialização com `WSAStartup`. Compilar nativamente no Windows sem adaptações resultou em erro ‘`arpa/inet.h: No such file or directory`’.
- **Serialização binária consistente:** garantir que ambos os lados usem a mesma ordenação de bytes e o mesmo tamanho total da mensagem (alinhamento/padding) — portanto optou-se por serializar campo a campo, usando `htonl/ntohl` e um buffer de bytes de tamanho fixo.
- **Robustez na leitura/escrita de sockets:** chamadas `send/recv` podem transferir menos bytes do que solicitados; foi necessário implementar loops que garantem envio/recebimento até completar o tamanho da mensagem.
- **Sincronização do protocolo:** cuidado para que ambos os lados utilizem o mesmo fluxo (`MSG_INIT` → `MSG_ACTION_REQ` → `MSG_ACTION_RES` → `MSG_BATTLE.RESULT` / `MSG_GAME_OVER`). Mensagens inesperadas devem ser ignoradas sem travar a conexão.
- **Tratamento de entradas do usuário:** uso de `scanf` se torna perigoso quando a entrada é inválida; foi adicionada limpeza do `stdin` para evitar loops de erro.
- **Regras de jogo ambíguas:** a contabilização de dano tinha casos que poderiam ser duplicados ou contabilizados duas vezes; foi necessário definir e documentar a ordem de prioridade (Hyper Jump tem prioridade absoluta; Shields bloqueiam Laser/Torpedo; Cloaking evita Torpedo mas não Laser; Torpedo vence Laser ao penalizar a parte que usou Laser).

3 Estratégias adotadas

As decisões e estratégias realizadas para enfrentar os desafios:

3.1 Serialização explícita

Para evitar problemas de *endianness* e padding, cada campo inteiro é convertido usando `htonl` para escrita e `ntohl` para leitura. A mensagem é construída em um buffer de tamanho fixo (`sizeof(BattleMessage)`) e enviada com loop até garantir envio completo.

3.2 Leitura robusta

Implementou-se um loop de leitura que só termina quando todos os bytes esperados foram recebidos ou ocorrer erro/fechamento de conexão.

3.3 Estrutura do protocolo e estados

Foi definido um fluxo simples de mensagens:

1. `MSG_INIT`: enviado pelo servidor no começo com HP iniciais;
2. loop: servidor envia `MSG_ACTION_REQ`; cliente responde `MSG_ACTION_RES`;
3. servidor envia `MSG_BATTLE_RESULT` ou `MSG_GAME_OVER`.

No cliente, uma flag `game_running` controla o loop principal e evita `breaks`, permitindo limpeza e mensagens de erro mais claras.

3.4 Compatibilidade Windows / WSL

Para manter desenvolvimento simples e reproduzível, foi adotada a seguinte opção:

- **Utilizado: WSL / ambiente Linux (Ubuntu)** — compila sem alterações; usa `make` e executáveis em `./bin`.

3.5 Validação e testes automáticos simples

Para testes rápidos sem digitação, usa-se:

```
# alimenta o client com 3 escolhas (0,1,4)
printf "0\n1\n4\n" | ./bin/client 127.0.0.1 12345
```

E para rodar o servidor em background:

```
./bin/server v4 12345 > server.log 2>&1 &
```

4 Possíveis melhorias

Lista de melhorias (priorizadas) que podem elevar robustez, segurança e funcionalidades:

1. **Protocolo com framing e tamanho dinâmico:** atualmente a mensagem tem tamanho fixo. Adotar um header com campo `length` permitiria mensagens variáveis, logs mais ricos e evitaria dependência de `sizeof(BattleMessage)`.
2. **Tratar timeouts e reconexão:** usar `select/poll` para detectar clientes inativos e aplicar timeouts em `recv/send`.
3. **Teste unitário e integração automatizada:** criar scripts que disparem múltiplos clientes e validem cenários (por exemplo, sequência que leva a vitória do cliente, empate, Hyper Jump).
4. **Separar lógica de jogo e I/O:** mover resolução de turno para um módulo separado (função que recebe duas ações e retorna danos/resultados). Facilita testes unitários.
5. **Melhor interface do cliente:** substituir o console por UI (ncurses ou web) para tornar interação mais amigável.
6. **Melhor tratamento de entradas e logs:** adicionar níveis de log (INFO/DEBUG/ERROR) e melhorar entradas para evitar travamentos.
7. **Versão nativa Windows testada:** fornecer binários e Makefile/testes para MSYS2/MinGW e para Visual Studio (caso o usuário precise rodar sem WSL).
8. **Segurança básica:** validar tamanhos, evitar strcpy sem limites e, se exposto em rede, considerar autenticação mínima ou limitação de conexões.

5 Conclusão

O trabalho implementa um protocolo simples mas completo de troca de mensagens binárias entre cliente e servidor, com atenção à portabilidade, robustez de I/O e regras do jogo. O principal desafio prático foi a portabilidade para Windows; a solução mais simples e recomendada para desenvolvimento é usar WSL/Ubuntu. As melhorias propostas permitem evoluir o projeto em direção a maior robustez, testabilidade e usabilidade.

No geral, o trabalho serviu como boa experiência para entender a dinâmica e o funcionamento da comunicação entre cliente e servidor.